

Creando mi OS

Encendiendo la computadora



2. Envío de señal de reset

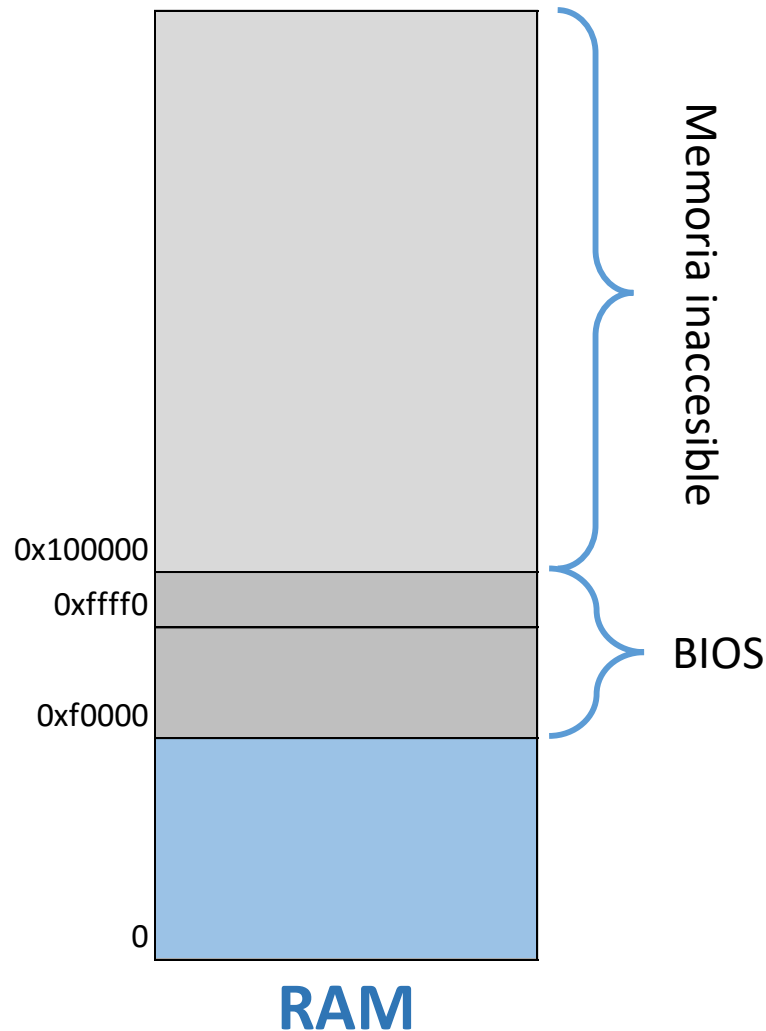


1. Power on reset



Pin de reset

Encendiendo la computadora: RESET



$$\begin{aligned}\text{Memoria física} &= (\text{CS} \ll 4) + \text{IP} \\ &= 0xffff0\end{aligned}$$

- Primera instrucción localizada en 0xffff0
- El procesador está en MODO REAL (compatibilidad con 8088/8086)
 - Limitado a un direccionamiento de 1 MiB (20 bits)
 - No hay protección ni niveles de privilegio
 - Acceso directo a toda la memoria
 - No hay multitarea
- La primera instrucción está al tope de la accesibilidad de la memoria
 - Luego hay que saltar a otra posición

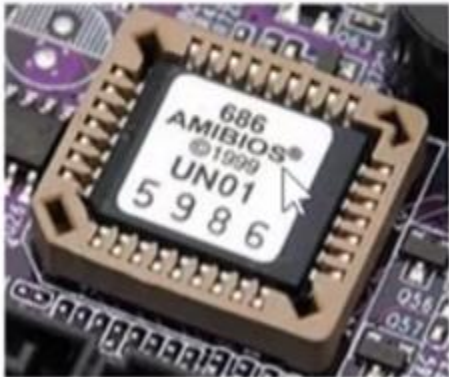
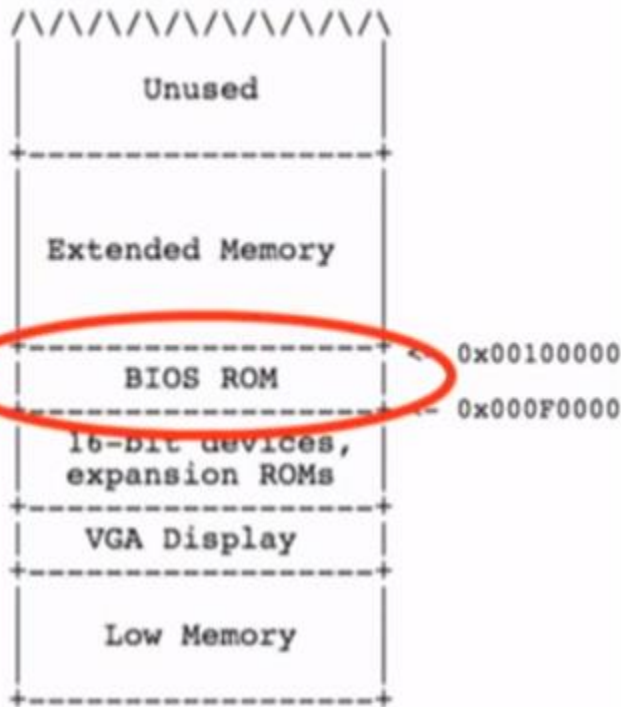
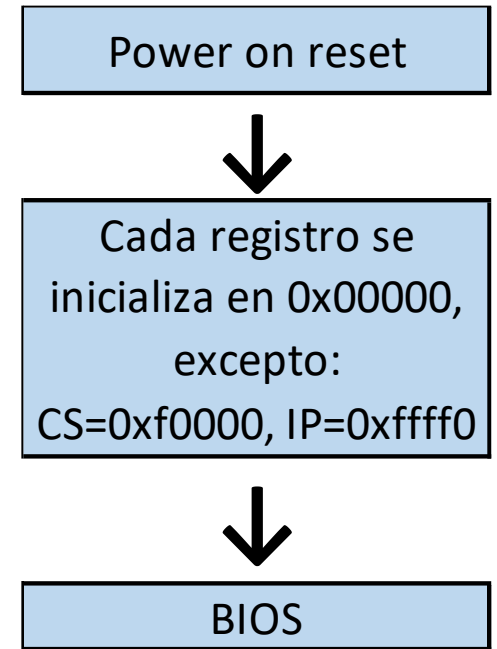
Power on reset



Cada registro se
inicializa en 0x00000,
excepto:
CS=0xf0000, IP=0xffff0

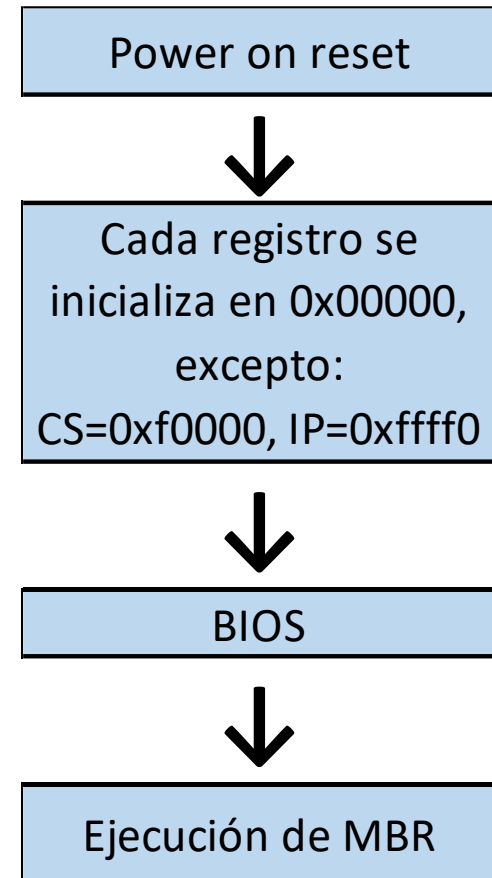
Encendiendo la computadora: BIOS

- Es un pequeño integrado conectado en la placa madre, donde también se encuentra el procesador (Flash, EPROM, EEPROM)
- El BIOS tiene software especializado para controlar las funciones básicas del sistema (firmware):
 - P.O.S.T. (Power on self test): Son rutinas de comprobación del estado del sistema.
 - Inicializa la tarjeta de video y otros dispositivos (algunos de esos otros dispositivos tienen su propio BIOS)
 - Visualiza la primera pantalla
 - Hace un test de la memoria
 - Establece los parámetros de la memoria
 - Configura los dispositivos plug & play
 - Asigna recursos (mapa de memoria) a los dispositivos (Canales DMA & IRQs)
 - **Identifica el dispositivo de arranque**
 - Lee el primer sector y lo localiza en RAM en 0x7c00
 - Salta a 0x7c00



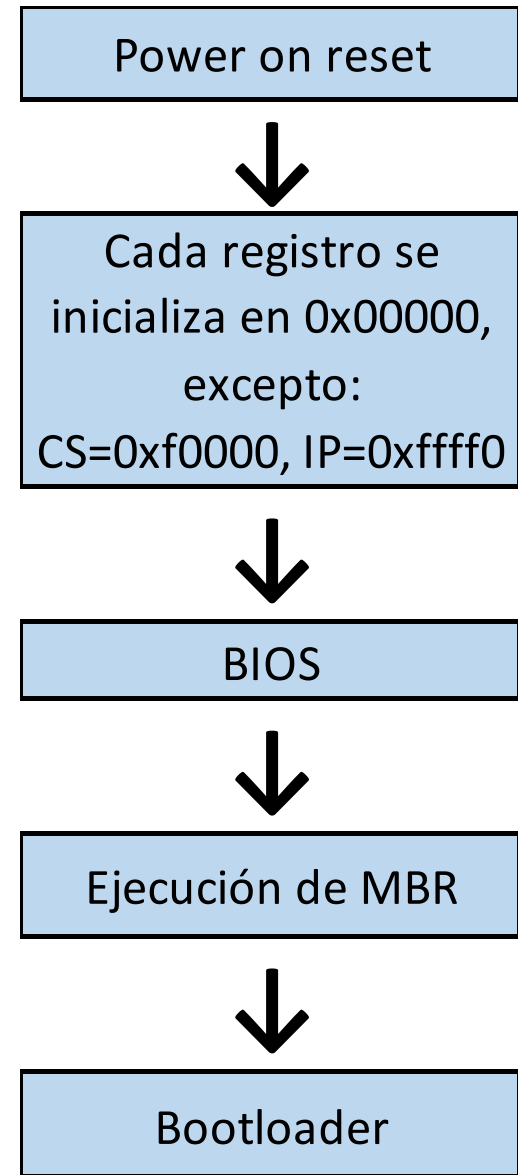
Encendiendo la computadora: MBR

- En el dispositivo de arranque, el sector 0 es llamado MBR (Master Boot Record)
- Contiene código para inicializar un SISTEMA OPERATIVO u otro bootloader
- Se copia el contenido de este sector a RAM en la posición **0x7c00** (memoria mapeada al BIOS) y comienza su ejecución.
- El tamaño de este código está establecido en **512 bytes**
 - 446 bytes de código de arranque
 - 64 bytes de información de la partición del dispositivo (16 bytes por partición, por tanto hay un máximo de 4 particiones)
 - 2 bytes de identificación de arranque (número mágico: byte 511=0x55, byte 512=0xAA)
- Generalmente, el código de MBR busca en la tabla de particiones y carga el bootloader (como Linux o Windows)
- O, carga directamente el SISTEMA OPERATIVO



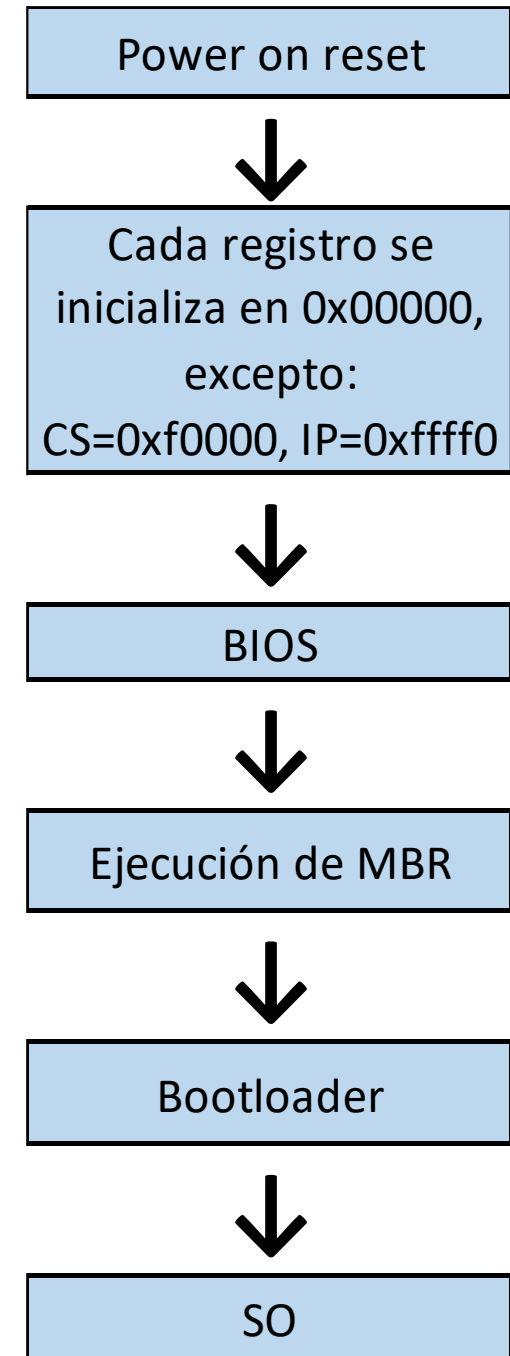
Encendiendo la computadora: bootloader

- Carga el SISTEMA OPERATIVO (Puede ocurrir que el usuario pueda seleccionar la carga de un SO en una lista de opciones)
- Otras rutinas que debe hacer:
 - Desactivar las interrupciones (no se quiere interferir con el proceso de arranque en esta etapa)
 - Configurar GDT (Global descriptor table) con el fin de definir las características de las distintas áreas de memoria utilizadas por los procesos, incluida la dirección base, el tamaño y los privilegios de acceso como la ejecutabilidad y la escritura (segmentos).
 - Cambiar de modo real a **modo protegido**.
 - Cargar de disco el resto del SO.



Encendiendo la computadora: SO

- Configurar la MEMORIA VIRTUAL
- Inicializar los vectores de interrupción
- Inicializar:
 - Timers
 - Monitores
 - Discos
 - Consolas o terminales
 - Sistemas de archivo
 - Otros procesadores (si existen)
- Cargar e iniciar procesos de usuario



El arranque

El siguiente código es el ejemplo más pequeño posible de arranque de código desde un disco.

```
ASM boot.s
1  ; boot.asm
2  hang:
3      jmp     hang
4
5      times 512-($-$$) db 0
6
```

La CPU se inicia en modo real y el BIOS carga este código en la dirección 0000:7c00.

"times 512-(\$-\$\$) db 0" es la forma como NASM dice que llene 512 bytes con ceros (200 en hexadecimal 512 en decimal).

Una vez que haya arrancado, y el cursor está parpadeando felizmente en una pantalla en blanco, el motor del disco se apagará y ahora puede presionar Ctrl-Alt-Supr para reiniciar. Esto se debe a que todavía se están generando interrupciones.

A menudo hay una firma de arranque (0xAA55) al final. Las versiones anteriores de algunos BIOS buscaron esto para identificar un sector de arranque en un disco. Evidentemente es innecesario hoy en día, a menos que esté ejecutando el código en un BIOS heredado, o en algún emulador como QEMU. Si es necesario, la última línea se reemplazaría con (o alguna versión de la misma):

```
ASM boot.s
1  ; boot.asm
2  hang:
3      jmp     hang
4
5      times 510-($-$$) db 0 ; ahora 2 bytes menos
6      db      0x55
7      db      0xAA
8
9      ;el tamaño del sector de arranque es de 512 bytes
```


El arranque

Si intentamos desactivar las interrupciones:

```
ASM boot.s
1  ;boot.asm
2  cli
3  hang:
4  jmp     hang
5
6  times 510-($-$$)    db  0
7  db  0x55
8  db  0xAA
```

Es posible que observe que el motor del disco no se apaga y que no puede reiniciar con Ctrl-Alt-Supr.

La eliminación del bucle y simplemente el relleno del sector con ceros generalmente hará que el BIOS produzca un error en el arranque. En la mayoría de las máquinas, dirá "*Sistema operativo no encontrado*".

Creación de imagen de disco

El código se ensambla en NASM y se copia en disquete (obsoleto), disco o memorias USB mediante partcopy, dd o debug. Para sistemas Unix (en nuestro caso usaremos Linux) usaremos dd.

Primero compilamos:

```
oaromero@EliteDesk:~$ nasm boot.asm -f bin -o boot.bin
```

Luego creamos la imagen:

```
oaromero@EliteDesk:~$ dd if=boot.bin of=boot.img
1+0 records in
1+0 records out
512 bytes copied, 0.0010269 s, 499 kB/s
```

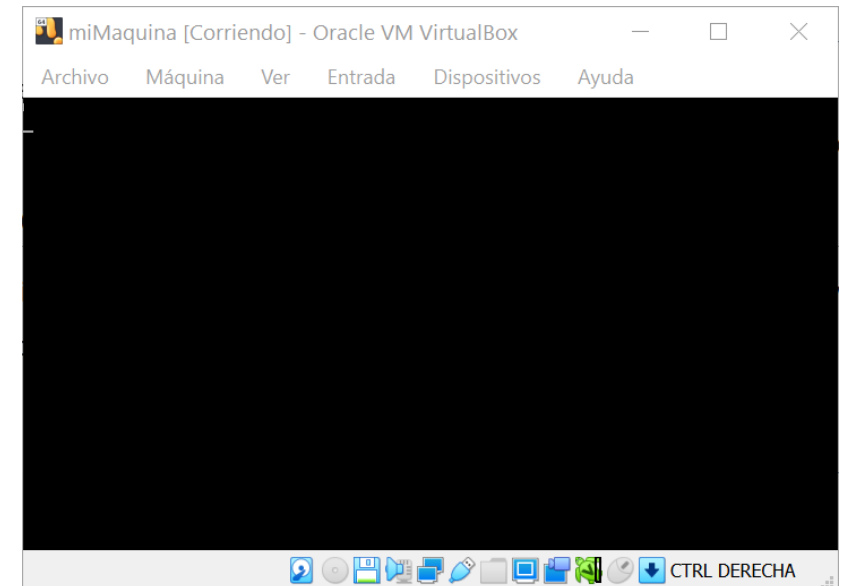
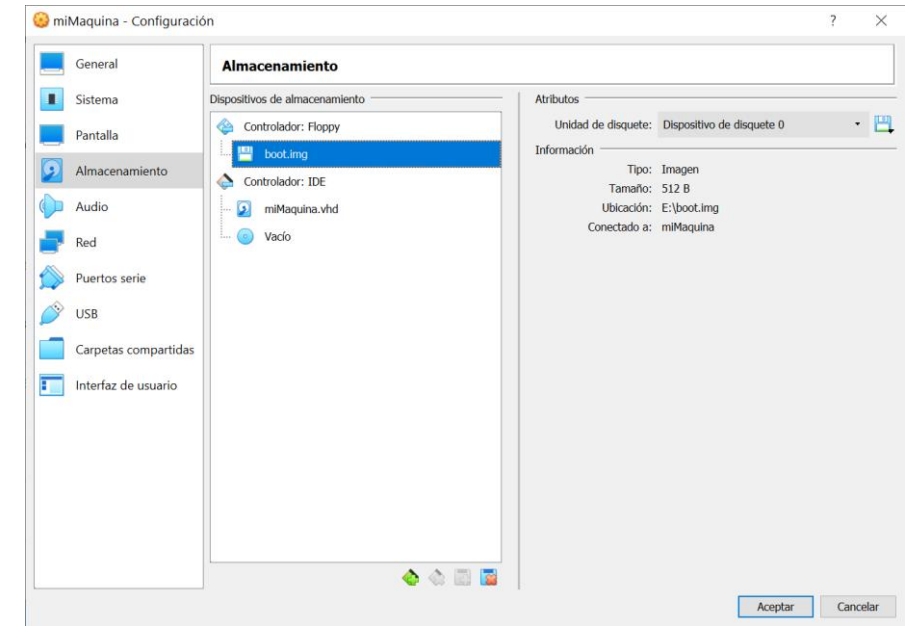
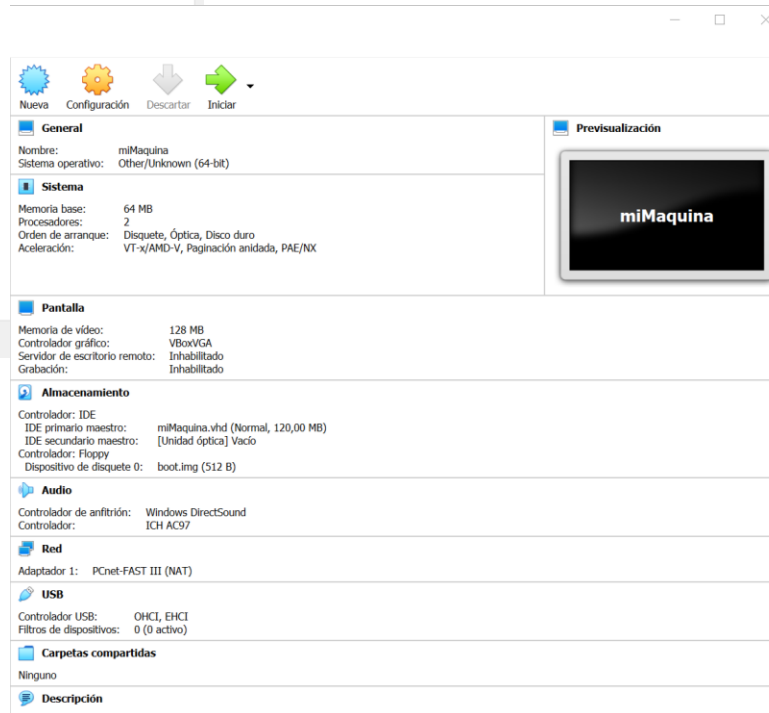
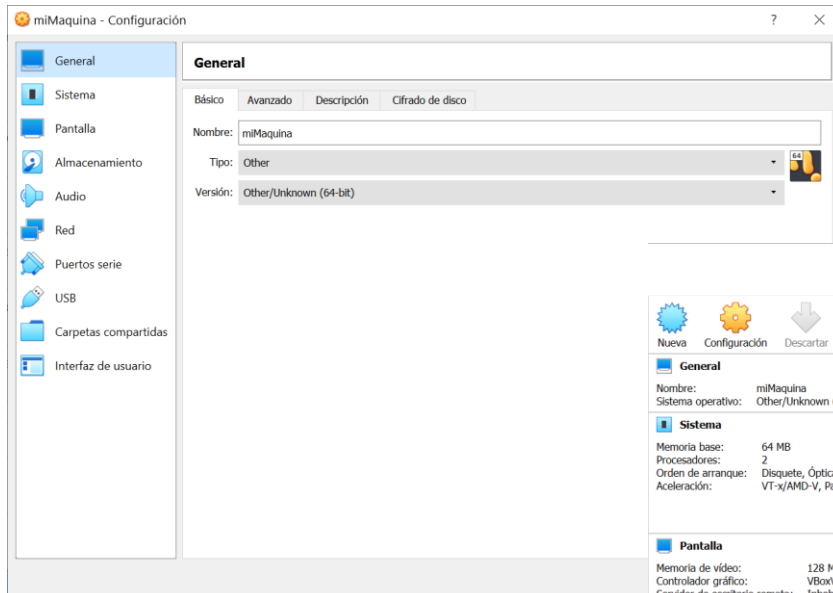
Para escribir en un disco duro o usb-stick, o en una imagen img utilice:

```
dd if=boot.bin of=/dev/fd0
dd if=boot.bin of=/dev/sda
dd if=boot.bin of=boot.img
```

El arranque

Si no tiene una máquina para probar el arranque con un USB o disquete, podemos emular el proceso.

Probando la imagen en VirtualBox



El arranque

Escribir un mensaje utilizando el BIOS

Resumen rápido:

1. El sector de arranque cargado por el BIOS es de 512 bytes
2. El código en el sector de arranque del disco es cargado por el BIOS en la dirección 0000:7c00
3. La máquina se inicia en **modo real**
4. Tenga en cuenta que la CPU siempre se interrumpe a menos que usted deshabilite las interrupciones con el comando CLI en el ensamblador.

Muchas (pero no todas) interrupciones del BIOS esperan que DS se llene con un valor de segmento del modo real. Esta es la razón por la que muchas interrupciones del BIOS no funcionarán en modo protegido. Por lo tanto, si desea utilizar int 10h/ah-0eh para imprimir en la pantalla, entonces se debe asegurar que su segmento:offset sea el correcto para que los caracteres se impriman.

Puesto que el desplazamiento puede ser mucho mayor que 16, hay muchos pares de segmentos y desplazamientos que apuntan a la misma dirección. Por ejemplo, algunos dicen que el gestor de arranque se carga en 0000:7C00, mientras que otros dicen 07C0:0000. De hecho, esta es la misma dirección: $16 * 0x0000 + 0x7C00$ a $16 * 0x07C0 + 0x0000$ a $0x7C00$.

```
1 ; boot.asm
2     mov     ax, 0x07c0
3     mov     ds, ax
4
5     mov     si, msg
6     cld     ;Borra el indicador DF en el registro EFLAGS.
7             ;Cuando el indicador DF se establece en 0,
8             ;las operaciones de cadena incrementan
9             ;los registros de índice (Este caso es SI)
10 ch_loop:
11     lodsb   ; Carga un byte de SI a AL
12             ; (Ya que estamos funcionando a 16 bits).
13     or      al, al ; str es 0?
14     jz      hang ; salimos
15     mov     ah, 0x0E
16     mov     bh, 0
17     int     0x10
18     jmp     ch_loop
19
20     hang:
21     jmp     hang
22
23     msg     db 'Arranque de mi OS', 13, 10, 0
24     times   510-($-$$) db 0
25     db      0x55
26     db      0xAA
27
```



El arranque

Escribir un mensaje utilizando el BIOS – mejorando el código

Aquí está la versión ORG. Esta vez, se accede a msg con el segmento 0. Tenga en cuenta que todavía debe indicar a DS qué ser, ya que puede inicializarse en cualquier valor.

```
1 ; boot.asm
2 [ORG 0x7c00]
3     xor ax, ax ; lo hacemos 0
4     mov ds, ax
5
6     mov si, msg
7     cld ;Borra el indicador DF en el registro EFLAGS.
8         ;Cuando el indicador DF se establece en 0,
9         ;las operaciones de cadena incrementan
10        ;los registros de índice (Este caso es SI)
11 ch_loop:
12     lodsb ; Carga un byte de SI a AL
13         ; (Ya que estamos funcionando a 16 bits).
14     or al, al ; str es 0?
15     jz hang ; salimos
16     mov ah, 0x0E
17     mov bh, 0
18     int 0x10
19     jmp ch_loop
20 hang:
21     jmp hang
22
23 msg db 'Arranque de mi OS', 13, 10, 0
24 times 510-($-$$) db 0
25 db 0x55
26 db 0xAA
```

```
1 ; boot.asm
2 [ORG 0x7c00]
3     xor ax, ax ; lo hacemos 0
4     mov ds, ax
5     cld ;Borra el indicador DF en el registro EFLAGS.
6         ;Cuando el indicador DF se establece en 0,
7         ;las operaciones de cadena incrementan
8         ;los registros de índice (Este caso es SI)
9     mov si, msg
10    call bios_print
11
12 hang:
13     jmp hang
14
15 msg db 'Arranque de mi OS', 13, 10, 0
16
17 bios_print:
18     lodsb ; Carga un byte de SI a AL
19         ; (Ya que estamos funcionando a 16 bits).
20     or al, al ; str es 0?
21     jz done ; salimos
22     mov ah, 0x0E
23     mov bh, 0
24     int 0x10
25     jmp bios_print
26 done:
27     ret
28
29     times 510-($-$$) db 0
30     db 0x55
31     db 0xAA
```

Para ahorrar espacio de escritura se usan "**procedimientos**"
Para separar del código se hace mediante CALL/RET.

El modo real

Es un modo simplista de 16 bits que está presente en todos los procesadores x86. El modo real fue el primer diseño para la familia x86 y fue utilizado por muchos sistemas operativos tempranos antes del nacimiento del modo protegido. Por motivos de compatibilidad, todos los procesadores x86 comienzan la ejecución en modo real.

Todos los sistemas operativos modernos (Windows, Linux, ...) se ejecutan en modo protegido, debido a las muchas limitaciones y problemas que presenta el modo real. Los sistemas operativos más antiguos (como DOS) los programas se ejecutaban en modo real porque era el único modo disponible en ese momento.

Contras:

- Menos de 1 MB de RAM está disponible para su uso.
- No hay protección de memoria basada en hardware (GDT), ni memoria virtual.
- No hay mecanismos de seguridad integrados para proteger contra aplicaciones maliciosas.
- La longitud predeterminada del operando de CPU es de solo 16 bits.
- Los modos de direccionamiento de memoria proporcionados son más restrictivos que otros modos de CPU.
- El acceso a más de 64k requiere el uso de registros de segmentos con los que es difícil trabajar.

Pros:

- El BIOS instala controladores de dispositivos para controlar los dispositivos y gestionar la interrupción.
- Las funciones del BIOS proporcionan a los sistemas operativos una colección avanzada de funciones api de bajo nivel.
- El acceso a la memoria es más rápido debido a la falta de tablas de descriptores para comprobar y los registros son más pequeños.

El modo real

Hay un poco más de 1 MB de memoria "direccionable" (incluyendo el área de memoria alta). La cantidad utilizable por un programa será mucho menor que 1 MB.

El acceso a la memoria se realiza mediante segmentación a través de un sistema *segmento : offset*(desplazamiento).

Hay seis registros de segmentos de 16 bits: CS, DS, ES, FS, GS y SS. Cuando se utilizan registros de segmentos, las direcciones se proporcionan con la siguiente notación (donde 'Segmento' es un valor en un registro de segmento y 'Offset' es un valor en un registro de direcciones):

12F3 : 4B27

^ ^

Segmento Offset

Los segmentos y desplazamientos están relacionados con direcciones físicas mediante la ecuación:

DirecciónFísica = Segmento * 16 + Offset

Por lo tanto, 12F3:4B27 corresponde a la dirección física 0x17A57. Cualquier dirección física se puede representar de varias maneras, con diferentes segmentos y desfases. Por ejemplo, la dirección física 0x210 puede ser 0020:0010, 0000:0210 o 0021:0000.

Empezar	Final	Tamaño	Descripción	Tipo	
Espacio de direcciones en modo real (el primer MiB)					
0x00000000	0x000003FF	1 KiB	Modo real IVT (Tabla vectorial de interrupción)	inutilizable en modo real	640 KiB RAM ("Memoria baja")
0x00000400	0x000004FF	256 bytes	BDA (área de datos del BIOS)	memoria utilizable	
0x00000500	0x00007BFF	casi 30 KiB	Memoria convencional		
0x00007C00	0x00007DFF	512 bytes	Su sistema operativo Bootsector		
0x00007E00	0x0007FFFF	480.5 KiB	Memoria convencional		
0x00080000	0x0009FFFF	128 KiB	EBDA (área de datos ampliada del BIOS)	utilizado parcialmente por la EBDA	
0x000A0000	0x000BFFFF	128 KiB	Memoria de visualización de vídeo	hardware asignado	Sistema de 384 KiB / Reservado ("Memoria Superior")
0x000C0000	0x000C7FFF	32 KiB (normalmente)	BIOS de vídeo	ROM y hardware mapeado / Shadow RAM	
0x000C8000	0x000EFFFF	160 KiB (normalmente)	Expansiones del BIOS		
0x000F0000	0x000FFFFF	64 KiB	BIOS de la placa base		